

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №7
Специальность ПО-13

Выполнил:
студент 3 курса
группы ПО-13
А.В. Король
Проверил:
А.Д.Кулик
01.04.2026

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
- Организовать ввод параметров для создания объектов (использовать экранные компоненты);
- Осуществить визуализацию графических примитивов

9) Создать классы Point и Line. Объявить список из n объектов класса Point. Для объекта класса Line определить, какие из объектов Point лежат на одной стороне от прямой линии и какие на другой. Реализовать ввод данных для объекта Line и случайное задание данных для объекта Point.

Выполнение:

Код программ:

```
import random
import tkinter as tk
from tkinter import messagebox
from tkinter import ttk

import matplotlib
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

matplotlib.use("TkAgg")

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class Line:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def side(self, point):
        return self.a * point.x + self.b * point.y + self.c

class App:
    def __init__(self, root):
        self.root = root

        self.root.title("Point and Line")
```

```

self.points = []

self.line = Line(1, -1, 0)

self.paused = False

self.after_id = None

self.build_ui()

self.fig, self.ax = plt.subplots(figsize=(6, 6))

self.canvas = FigureCanvasTkAgg(
    self.fig,
    master=self.plot_frame,
)

self.canvas.get_tk_widget().pack(
    fill="both",
    expand=True,
)

self.root.protocol(
    "WM_DELETE_WINDOW",
    self.on_close,
)

self.update_plot()

def build_ui(self):
    control = ttk.Frame(self.root)

    control.pack(
        side="left",
        fill="y",
        padx=10,
        pady=10,
    )

    ttk.Label(
        control,
        text="Points count",
    ).pack()

    self.n_entry = ttk.Entry(control)

    self.n_entry.insert(0, "30")

    self.n_entry.pack(pady=5)

    ttk.Button(
        control,
        text="Generate points",
        command=self.generate_points,

```

```

).pack(pady=5)

ttk.Label(
    control,
    text="Line: Ax + By + C = 0",
).pack(pady=10)

self.a_entry = ttk.Entry(control)

self.b_entry = ttk.Entry(control)

self.c_entry = ttk.Entry(control)

for label, entry, value in [
    ("A", self.a_entry, "1"),
    ("B", self.b_entry, "-1"),
    ("C", self.c_entry, "0"),
]:
    ttk.Label(
        control,
        text=label,
    ).pack()

    entry.insert(0, value)

    entry.pack(pady=2)

ttk.Button(
    control,
    text="Update line",
    command=self.update_line,
).pack(pady=10)

ttk.Button(
    control,
    text="Pause",
    command=self.toggle_pause,
).pack(pady=5)

ttk.Button(
    control,
    text="Screenshot",
    command=self.save_screenshot,
).pack(pady=5)

self.plot_frame = ttk.Frame(self.root)

self.plot_frame.pack(
    side="right",
    fill="both",
    expand=True,
)

def generate_points(self):

```

```

try:
    count = int(self.n_entry.get())

except ValueError:
    messagebox.showerror(
        "Error",
        "Invalid number",
    )

    return

self.points = [
    Point(
        random.uniform(-10, 10),
        random.uniform(-10, 10),
    )
    for _ in range(count)
]

def update_line(self):
    try:
        a = float(self.a_entry.get())

        b = float(self.b_entry.get())

        c = float(self.c_entry.get())

    except ValueError:
        messagebox.showerror(
            "Error",
            "Invalid coefficients",
        )

        return

    self.line = Line(a, b, c)

def toggle_pause(self):
    self.paused = not self.paused

def save_screenshot(self):
    self.fig.savefig("screenshot.png")

    messagebox.showinfo(
        "Saved",
        "Screenshot saved",
    )

def update_plot(self):
    if not self.root.winfo_exists():
        return

    if not self.paused:
        self.ax.clear()

```

```

self.ax.grid(True)

self.ax.set_title("Point-Line")

a = self.line.a

b = self.line.b

c = self.line.c

x_values = [-10, 10]

if b != 0:
    y_values = [(-a * x - c) / b for x in x_values]

else:
    x_values = [-c / a] * 2

    y_values = [-10, 10]

self.ax.plot(
    x_values,
    y_values,
    "k-",
)

left = []

right = []

on_line = []

for point in self.points:
    side = self.line.side(point)

    if side > 0:
        right.append(point)

    elif side < 0:
        left.append(point)

    else:
        on_line.append(point)

self.ax.scatter(
    [p.x for p in left],
    [p.y for p in left],
    color="red",
    label="Left",
)

self.ax.scatter(
    [p.x for p in right],

```

```

        [p.y for p in right],
        color="blue",
        label="Right",
    )

    self.ax.scatter(
        [p.x for p in on_line],
        [p.y for p in on_line],
        color="green",
        label="On line",
    )

    self.ax.legend()

    self.canvas.draw()

    self.after_id = self.root.after(
        200,
        self.update_plot,
    )

def on_close(self):
    if self.after_id is not None:
        self.root.after_cancel(self.after_id)

    self.root.destroy()

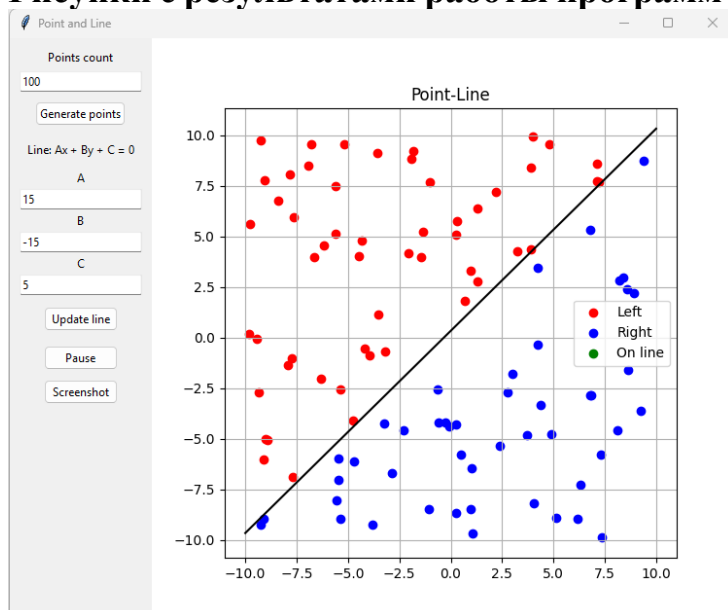
```

```
root = tk.Tk()
```

```
app = App(root)
```

```
root.mainloop()
```

Рисунки с результатами работы программ



Задание 2. Реализовать построение заданного типа фрактала по варианту

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

9) Остров Минковского

Выполнение:

Код программ:

```
import tkinter as tk
from tkinter import ttk

import matplotlib
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

matplotlib.use("TkAgg")

def minkowski_curve(
    x1,
    y1,
    x2,
    y2,
    depth,
    points,
):
    if depth == 0:
        points.append((x1, y1))

        return

    dx = (x2 - x1) / 4

    dy = (y2 - y1) / 4

    points_list = [
        (x1, y1),
        (x1 + dx, y1 + dy),
        (x1 + dx - dy, y1 + dy + dx),
        (x1 + 2 * dx - dy, y1 + 2 * dy + dx),
        (x1 + 2 * dx, y1 + 2 * dy),
        (x1 + 3 * dx, y1 + 3 * dy),
        (x1 + 3 * dx + dy, y1 + 3 * dy - dx),
        (x1 + 4 * dx + dy, y1 + 4 * dy - dx),
        (x2, y2),
    ]

    for index in range(len(points_list) - 1):
        minkowski_curve(
            points_list[index][0],
            points_list[index][1],
            points_list[index + 1][0],
            points_list[index + 1][1],
            depth - 1,
```



```
        points,  
    )
```

```
def minkowski_island(depth):  
    points = []
```

```
    minkowski_curve(  
        0,  
        0,  
        1,  
        0,  
        depth,  
        points,  
    )
```

```
    minkowski_curve(  
        1,  
        0,  
        1,  
        1,  
        depth,  
        points,  
    )
```

```
    minkowski_curve(  
        1,  
        1,  
        0,  
        1,  
        depth,  
        points,  
    )
```

```
    minkowski_curve(  
        0,  
        1,  
        0,  
        0,  
        depth,  
        points,  
    )
```

```
    points.append((0, 0))
```

```
    return points
```

```
class App:
```

```
    def __init__(self, root):  
        self.root = root
```

```
        self.root.title("Minkowski Island")
```

```

self.depth = 1

self.build_ui()

self.fig, self.ax = plt.subplots(figsize=(6, 6))

self.canvas = FigureCanvasTkAgg(
    self.fig,
    master=self.plot_frame,
)

self.canvas.get_tk_widget().pack(
    fill="both",
    expand=True,
)

self.root.protocol(
    "WM_DELETE_WINDOW",
    self.on_close,
)

self.draw_fractal()

def build_ui(self):
    control = ttk.Frame(self.root)

    control.pack(
        side="left",
        fill="y",
        padx=10,
        pady=10,
    )

    ttk.Label(
        control,
        text="Depth",
    ).pack()

    self.depth_entry = ttk.Entry(control)

    self.depth_entry.insert(0, "1")

    self.depth_entry.pack(pady=5)

    ttk.Button(
        control,
        text="Draw",
        command=self.update_depth,
    ).pack(pady=10)

    self.plot_frame = ttk.Frame(self.root)

    self.plot_frame.pack(
        side="right",

```

```

        fill="both",
        expand=True,
    )

def update_depth(self):
    try:
        self.depth = int(self.depth_entry.get())

    except ValueError:
        self.depth = 1

    self.draw_fractal()

def draw_fractal(self):
    self.ax.clear()

    points = minkowski_island(self.depth)

    x_values = [point[0] for point in points]

    y_values = [point[1] for point in points]

    self.ax.plot(
        x_values,
        y_values,
        color="blue",
        linewidth=1,
    )

    self.ax.set_aspect("equal")

    self.ax.set_title(f"Minkowski Island depth {self.depth}")

    self.ax.axis("off")

    self.canvas.draw()

def on_close(self):
    plt.close(self.fig)

    self.root.destroy()

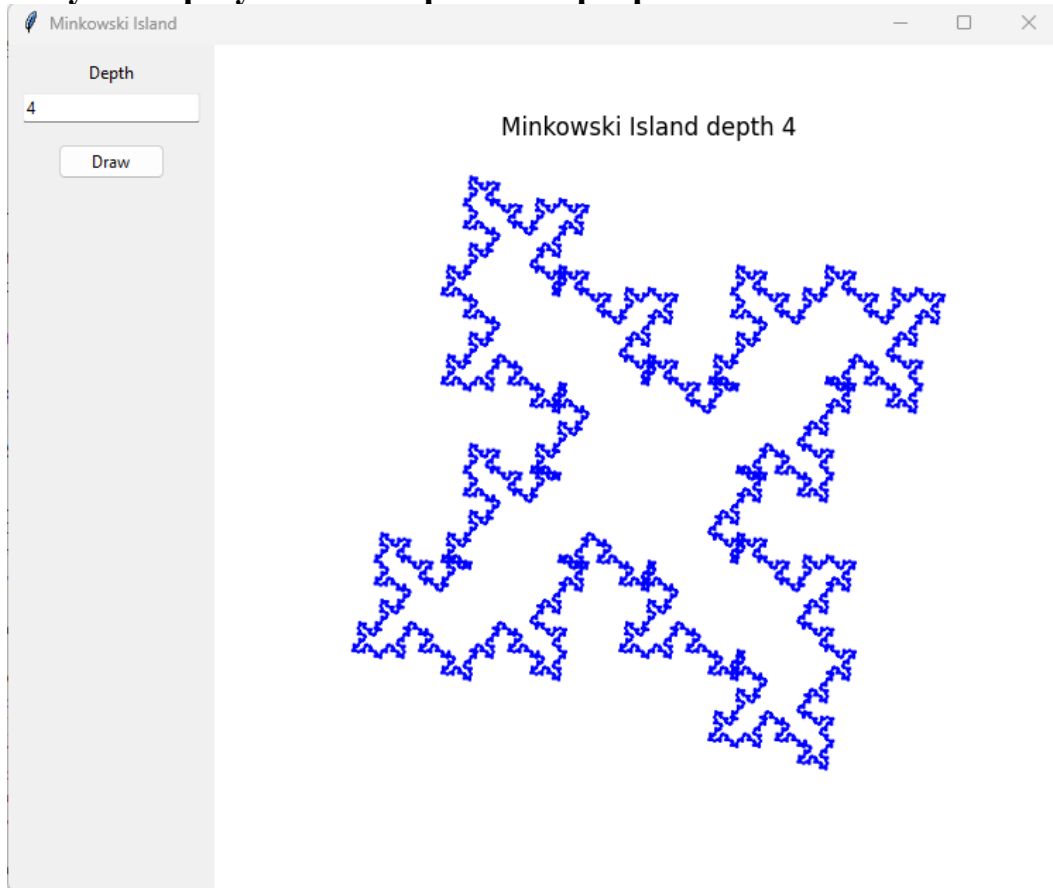
root = tk.Tk()

app = App(root)

root.mainloop()

```

Рисунки с результатами работы программ



Вывод: освоил возможности языка программирования Python в разработке оконных приложений